

FaceTracking Automatico

Telecamera smart con riconoscimento facciale e inseguimento servo-motorizzato

Michele Valiati

Fabian Shkina

Federico Chizzoli

Classe 4IB · May 2026

Linguaggi	C++ (Arduino) · Python
Hardware	ESP32-CAM · Arduino · 2× Servo SG90
Protocollo	HTTP MJPEG Stream (Wi-Fi)
Vision	MediaPipe BlazeFace · OpenCV
Controllo	PID con smoothing su finestra mobile

1. Descrizione del Progetto

FaceTracking Automatico è una telecamera smart che combina un modulo ESP32-CAM, due servo-motori e un'applicazione Python per riconoscere in tempo reale fino a tre volti nell'inquadratura e mantenere automaticamente il soggetto al centro del frame. Il sistema è pensato per applicazioni di videosorveglianza intelligente, streaming automatizzato e robotica educativa.

Obiettivi principali:

- Trasmissione video MJPEG dal modulo ESP32-CAM via Wi-Fi verso un PC.
- Rilevamento facciale lato PC con il modello BlazeFace di MediaPipe.
- Calcolo del centroide tra tutti i volti rilevati (max 3).
- Correzione dell'orientamento della telecamera mediante due servo (pan/tilt) controllati da un algoritmo PID con smoothing.
- Comunicazione seriale tra PC e scheda Arduino per il movimento fisico dei servo.

2. Architettura del Sistema

Il progetto è suddiviso in tre componenti software indipendenti che comunicano tra loro attraverso Wi-Fi e porta seriale USB.

Componente	File	Responsabilità
ESP32-CAM Firmware	ESP32_Cam_HTTP.cpp	Acquisisce i frame JPEG e li trasmette via HTTP MJPEG stream sulla porta 80.
Arduino Servo Driver	main.cpp	Riceve angoli pan/tilt dalla seriale e aggiorna i due servo SG90.
PC Vision & Control	FaceTracking_Servo_HTTP.py	Legge lo stream, rileva i volti, esegue il PID e invia comandi seriali all'Arduino.

Flusso dati:

ESP32-CAM → (HTTP MJPEG / Wi-Fi) → Python → (OpenCV + MediaPipe) → PID → (Seriale USB 115200 baud) → Arduino → Servo Pan/Tilt

3. Componenti Hardware

Componente	Descrizione	Note
ESP32-CAM	Modulo Wi-Fi con fotocamera OV2640 integrata	QVGA 320×240 JPEG, PSRAM per double buffering
Arduino (Uno/Nano)	Microcontrollore per gestione servo via PWM	Collegato al PC via USB seriale

Servo Pan — SG90	Rotazione orizzontale (0°–180°)	Pin D6, impulso 500–2400 µs
Servo Tilt — SG90	Rotazione verticale (0°–180°)	Pin D5, impulso 500–2400 µs
PC host	Esegue Python con OpenCV e MediaPipe	Connesso alla stessa rete Wi-Fi dell'ESP32

4. ESP32-CAM Firmware (ESP32_Cam_HTTP.cpp)

Il firmware è scritto con Arduino framework (PlatformIO/Arduino IDE). All'avvio tenta di connettersi alla rete Wi-Fi specificata come STA; se entro 10 secondi la connessione fallisce, attiva un Access Point di fallback (SSID: ESPCAM_SVC).

Configurazione fotocamera:

- Risoluzione QVGA (320×240), formato JPEG con qualità 10.
- Double frame buffer in PSRAM con modalità GRAB_LATEST per latenza minima.
- Gain ceiling 4×, sharpness +2, denoise attivo.
- Clock XCLK a 20 MHz.

Server HTTP:

- Porta 80, max 3 socket aperti contemporaneamente (MAX_CLIENTS + 1).
- Endpoint GET /stream — risposta multipart/x-mixed-replace;boundary=frame.
- Header CORS Access-Control-Allow-Origin: * per compatibilità browser.
- Semaforo FreeRTOS per accesso esclusivo alla fotocamera.
- Frequenza target ~25 FPS (delay 20 ms per frame).

Snippet — stream handler:

```
while (true) {
    camera_fb_t *fb = esp_camera_fb_get();
    if (!fb) { vTaskDelay(1); continue; }
    httpd_resp_send_chunk(req, PART_HEADER, part_len);
    httpd_resp_send_chunk(req, (const char*)fb->buf, fb->len);
    esp_camera_fb_return(fb);
    vTaskDelay(pdMS_TO_TICKS(20)); // ~25 FPS
}
```

5. Arduino Servo Driver (main.cpp)

Firmware minimale per Arduino che legge dalla seriale coppie di angoli nel formato **pan,tilt\n** e comanda i due servo via libreria Servo.

Protocollo seriale:

- Baud rate 115200, terminatore di riga \n.
- Formato messaggio: due interi separati da virgola, es. 90,75.

- Il primo valore è l'angolo pan (pin D6), il secondo il tilt (pin D5).
- Range valido: 0°–180° con impulso PWM calibrato 500–2400 μ s.

Snippet — parsing seriale:

```
while (Serial.available()) {
  char byte = Serial.read();
  if (byte != '\n') {
    messaggio += byte;
  } else {
    int virgola = messaggio.indexOf(',');
    panAngle = messaggio.substring(0, virgola).toInt();
    tiltAngle = messaggio.substring(virgola + 1).toInt();
    panServo.write(panAngle);
    tiltServo.write(tiltAngle);
    messaggio = "";
  }
}
```

6. Applicazione Python (FaceTracking_Servo_HTTP.py)

6.1 Lettura dello stream MJPEG

Un thread dedicato (stream_reader) apre la connessione HTTP all'ESP32 e analizza byte per byte il flusso JPEG cercando i marcatori SOI (0xFF 0xD8) e EOI (0xFF 0xD9). Il frame completato viene decodificato con cv2.imdecode e depositato in una variabile condivisa protetta da Lock.

6.2 Rilevamento facciale

Un secondo thread (detection_worker) legge il frame corrente ogni DETECT_SKIP=4 iterazioni e lo passa al detector MediaPipe BlazeFace. Vengono selezionati al massimo 3 volti con lo score di confidenza più alto (soglia minima 0.55). Le bounding box risultanti sono scritte nella lista condivisa latest_faces.

6.3 Algoritmo PID

Il loop principale calcola il centroide tra tutti i volti rilevati. L'errore rispetto al centro del frame alimenta due controllori PID indipendenti (pan e tilt). Una dead zone di ± 15 pixel evita oscillazioni quando il volto è già centrato.

Parametro	Valore	Descrizione
Kp	0.04	Guadagno proporzionale
Ki	0.001	Guadagno integrale (anti-windup)
Kd	0.008	Guadagno derivativo (smorzamento)
Dead zone	± 15 px	Reset integrale e derivativo nella zona morta
Smooth window	13 frame	Media mobile per smoothing dell'angolo servo

Servo Hz	30 Hz	Frequenza massima invio comandi seriali
----------	-------	---

6.4 HUD e visualizzazione

- Rettangolo verde attorno a ogni volto con percentuale di confidenza.
- Linea colorata dal centro frame al centro di ogni volto rilevato.
- Colore linea: rosso (1 volto), blu (2 volti), giallo-ciano (3 volti).
- Mirino centrale grigio fisso per riferimento visivo.

6.5 Timeout e recovery

Se nessun volto viene rilevato per 3 secondi (NO_FACE_TIMEOUT), i servo tornano gradualmente alla posizione centrale (90°, 90°) tramite lo stesso meccanismo di smoothing. Il PID viene resettato a ogni assenza di rilevamento per evitare l'accumulo integrale.

7. Configurazione e Avvio

Dipendenze Python:

```
pip install opencv-python mediapipe pyserial numpy
```

Parametri principali da configurare in FaceTracking_Servo_HTTP.py:

Variabile	Default	Descrizione
ESP32_IP	10.42.0.184	Indirizzo IP assegnato all'ESP32 dalla rete
SERIAL_PORT	/dev/ttyUSB0	Porta seriale Arduino (Linux). Su Windows: COM3, ecc.
IMAGE_ROTATION	-2	Gradi di rotazione applicati a ogni frame (correzione montaggio)
MODEL_PATH	face.tflite	Path al modello BlazeFace (scaricato automaticamente se mancante)

Ordine di avvio:

1. Caricare ESP32_Cam_HTTP.cpp sull'ESP32-CAM con PlatformIO o Arduino IDE.
2. Caricare main.cpp sull'Arduino (Uno/Nano).
3. Verificare che ESP32 e PC siano sulla stessa rete Wi-Fi e annotare l'IP dell'ESP32.
4. Aggiornare ESP32_IP e SERIAL_PORT nel file Python.
5. Avviare lo script: python FaceTracking_Servo_HTTP.py
6. Premere 'q' nella finestra OpenCV per terminare.

8. Struttura dei File

```
FaceTracking-Automatico/
```

```
■■■ ESP32_Cam_HTTP.cpp # Firmware ESP32-CAM
■■■ main.cpp # Firmware Arduino servo driver
■■■ FaceTracking_Servo_HTTP.py # Applicazione PC
■■■ face.tflite # Modello BlazeFace (auto-download)
```

9. Note e Possibili Sviluppi Futuri

- Aggiungere autenticazione Wi-Fi WPA2 Enterprise per deployment in reti scolastiche.
- Sostituire la comunicazione seriale con BLE o Wi-Fi diretto verso l'ESP32 per eliminare il cavo USB tra PC e Arduino.
- Integrare il controllo servo direttamente nel firmware ESP32 per rendere il sistema completamente standalone (senza PC host).
- Supporto multi-camera con switch automatico della sorgente video.
- Esportazione dei dati di tracciamento (timestamp, posizione volto) su file CSV.
- Web UI di configurazione per modificare i parametri PID senza riavviare lo script.